

File Organization Techniques

```
#include <stdio.h>
#include <string.h>

#define MAX 10

/* =====
A) SINGLE LEVEL DIRECTORY
===== */

struct { char name[20]; int block; } sld[MAX];
int sld_n = 0;

void sld_create(const char *name, int block) {
    for (int i = 0; i < sld_n; i++)
        if (!strcmp(sld[i].name, name)) { printf("SLD: '%s' exists\n", name); return; }
    strcpy(sld[sld_n].name, name); sld[sld_n++].block = block;
    printf("[SLD] Created '%s' at block %d\n", name, block);
}

void sld_delete(const char *name) {
    for (int i = 0; i < sld_n; i++)
        if (!strcmp(sld[i].name, name)) {
            sld[i] = sld[--sld_n];
            printf("[SLD] Deleted '%s'\n", name); return;
        }
    printf("SLD: '%s' not found\n", name);
}

void sld_show() {
    printf("\n-- Single Level Directory --\n");
    printf("%-15s %s\n", "File", "Block");
    for (int i = 0; i < sld_n; i++)
        printf("%-15s %d\n", sld[i].name, sld[i].block);
}

/* =====
B) TWO LEVEL DIRECTORY
===== */

struct {
    char user[20];
    struct { char name[20]; int block; } files[MAX];
    int fc;
```

```

} tld[MAX];
int tld_n = 0;

int tld_user(const char *user) {
    for (int i = 0; i < tld_n; i++) if (!strcmp(tld[i].user, user)) return i;
    strcpy(tld[tld_n].user, user); tld[tld_n].fc = 0;
    printf("[TLD] New user '%s'\n", user);
    return tld_n++;
}

void tld_create(const char *user, const char *name, int block) {
    int u = tld_user(user);
    for (int i = 0; i < tld[u].fc; i++)
        if (!strcmp(tld[u].files[i].name, name)) { printf("TLD: '%s/%s' exists\n", user, name); return; }
    strcpy(tld[u].files[tld[u].fc].name, name);
    tld[u].files[tld[u].fc++].block = block;
    printf("[TLD] Created '%s/%s' at block %d\n", user, name, block);
}

void tld_show() {
    printf("\n-- Two Level Directory --\n");
    for (int i = 0; i < tld_n; i++) {
        printf("User: %s\n", tld[i].user);
        for (int j = 0; j < tld[i].fc; j++)
            printf("  %-15s block %d\n", tld[i].files[j].name, tld[i].files[j].block);
    }
}

/* =====
C) HIERARCHICAL DIRECTORY
===== */

#define MAXNODES 30
#define MAXCHILD 10

struct Node {
    char name[20];
    int is_dir;
    int children[MAXCHILD];
    int child_n;
    int block; // for files
} nodes[MAXNODES];
int node_n = 0;

int new_node(const char *name, int is_dir, int block) {

```

```

    strcpy(nodes[node_n].name, name);
    nodes[node_n].is_dir = is_dir;
    nodes[node_n].child_n = 0;
    nodes[node_n].block = block;
    return node_n++;
}

// find child by name under parent, return index or -1
int find_child(int parent, const char *name) {
    for (int i = 0; i < nodes[parent].child_n; i++) {
        int c = nodes[parent].children[i];
        if (!strcmp(nodes[c].name, name)) return c;
    }
    return -1;
}

// path like "home/user/file.txt"; is_dir=0 for file
void hier_create(int root, const char *path, int is_dir, int block) {
    char buf[100]; strcpy(buf, path);
    char *tok = strtok(buf, "/");
    int cur = root;
    while (tok) {
        char *next_tok = strtok(NULL, "/");
        int child = find_child(cur, tok);
        if (!next_tok) { // last component = create it
            if (child != -1) { printf("HIER: '%s' exists\n", tok); return; }
            int n = new_node(tok, is_dir, block);
            nodes[cur].children[nodes[cur].child_n++] = n;
            printf("[HIER] Created %s '%s'\n", is_dir ? "dir" : "file", path);
            return;
        }
        if (child == -1) { // auto-create intermediate dir
            child = new_node(tok, 1, 0);
            nodes[cur].children[nodes[cur].child_n++] = child;
        }
        cur = child; tok = next_tok;
    }
}

void hier_print(int node, int depth) {
    for (int i = 0; i < depth; i++) printf(" ");
    printf("%s %s\n", nodes[node].is_dir ? "[D]" : "[F]", nodes[node].name);
    for (int i = 0; i < nodes[node].child_n; i++)
        hier_print(nodes[node].children[i], depth + 1);
}

```

```

}

/* =====
MAIN
===== */
int main() {

    printf("1BM24CS361, Nitya R Swadi\n\n");
    // A) Single Level
    printf("=== A) SINGLE LEVEL DIRECTORY ===\n");
    sld_create("file1.txt", 5); sld_create("file2.txt", 9);
    sld_create("photo.png", 3); sld_create("file1.txt", 7); // duplicate test
    sld_delete("file2.txt");
    sld_show();

    // B) Two Level
    printf("\n=== B) TWO LEVEL DIRECTORY ===\n");
    tld_create("alice", "notes.txt", 10); tld_create("alice", "photo.jpg", 14);
    tld_create("bob", "code.c", 20); tld_create("bob", "notes.txt", 22);
    tld_show();

    // C) Hierarchical
    printf("\n=== C) HIERARCHICAL DIRECTORY ===\n");
    int root = new_node("/", 1, 0);
    hier_create(root, "home/alice/notes.txt", 0, 10);
    hier_create(root, "home/alice/photo.jpg", 0, 14);
    hier_create(root, "home/bob/code.c", 0, 20);
    hier_create(root, "etc/config.cfg", 0, 30);
    hier_create(root, "etc/hosts", 0, 31);
    printf("\nDirectory Tree:\n");
    hier_print(root, 0);
}

```

OUTPUT:

```
"C:\Users\Nitya\OneDrive\De  X + v
1BM24CS361, Nitya R Swadi

=== A) SINGLE LEVEL DIRECTORY ===
[SLD] Created 'file1.txt' at block 5
[SLD] Created 'file2.txt' at block 9
[SLD] Created 'photo.png' at block 3
SLD: 'file1.txt' exists
[SLD] Deleted 'file2.txt'

-- Single Level Directory --
File          Block
file1.txt     5
photo.png     3

=== B) TWO LEVEL DIRECTORY ===
[TLD] New user 'alice'
[TLD] Created 'alice/notes.txt' at block 10
[TLD] Created 'alice/photo.jpg' at block 14
[TLD] New user 'bob'
[TLD] Created 'bob/code.c' at block 20
[TLD] Created 'bob/notes.txt' at block 22

-- Two Level Directory --
User: alice
  notes.txt      block 10
  photo.jpg     block 14
User: bob
  code.c        block 20
  notes.txt     block 22

=== C) HIERARCHICAL DIRECTORY ===
[HIER] Created file 'home/alice/notes.txt'
[HIER] Created file 'home/alice/photo.jpg'
[HIER] Created file 'home/bob/code.c'
[HIER] Created file 'etc/config.cfg'
```

=== C) HIERARCHICAL DIRECTORY ===

[HIER] Created file 'home/alice/notes.txt'

[HIER] Created file 'home/alice/photo.jpg'

[HIER] Created file 'home/bob/code.c'

[HIER] Created file 'etc/config.cfg'

[HIER] Created file 'etc/hosts'

Directory Tree:

[D] /

 [D] home

 [D] alice

 [F] notes.txt

 [F] photo.jpg

 [D] bob

 [F] code.c

 [D] etc

 [F] config.cfg

 [F] hosts

Process returned 0 (0x0) execution time : 0.063 s

Press any key to continue.

|